Enterprise Bot

DevOps Take-Home Assignment

Contents

1. Project Overview 3

2. Development Environment 3

2.1 AWS EC2 Instance Setup 3

2.2 Project Transfer and Extraction 4

2.3 Development Tools Installation 5

3. Kubernetes Cluster Setup 5

4. Part 1 – Python Application and Docker 6

4.1 Application Structure 6

4.2 Python Flask Application 6

4.3 Application Dependencies 6

4.4 Dockerfile 7

4.5 Docker Ignore 7

4.6 Docker Image Build 7

4.7 Container Deployment 8

4.8 Application Endpoint Verification 8

5. Part 2 – Helm and Kubernetes Deployment 9

5.1 Helm Chart 9

5.2 Chart Validation 9

5.3 Kubernetes Resource Inspection 9

6. Part 3 – Deployment Automation 10

6.1 Deployment Script (setup.sh) 10

6.2 Deployment Verification 12

7. Part 4 – Kubernetes Troubleshooting 13

7.1 Overview 13

7.2 Issues Investigated and Configuration Changes 13

7.3 Reporter Troubleshooting Evidence 14

7.4 Kubernetes API Connectivity Tests 14

7.5 Verification Status 15

7.6 Troubleshooting Conclusion 15

8. Part 5 – Ingress to Gateway API Migration Strategy 15

9. Part 6 – Production Readiness Considerations 16

9.1 Resource Management....16
9.2 Health Checks....16
9.3 Security....16
9.4 Monitoring....16
9.5 High Availability....17
9.6 Deployment Strategy....17
9.7 Further Improvements....17

10. Git and GitHub.... 17

11. Final Project Status.... 18

12. Conclusion.... 18

1. Project Overview

The objective of this assignment was to build and deploy a containerized Python application using Docker, Kubernetes, and Helm. The assignment also involved automating deployment operations and troubleshooting an intentionally broken Kubernetes environment.

The work covered:

- Preparing an Ubuntu-based AWS EC2 development environment.
- Developing a Python Flask application.
- Building and running a Docker image.
- Creating and managing a Kubernetes cluster using Kind.
- Deploying applications using Helm.
- Automating deployment operations using shell scripts.
- Troubleshooting Kubernetes workloads, configurations, permissions, and resource issues.
- Documenting a migration strategy from Ingress to Gateway API.

2. Development Environment

Component	Technology
Cloud Platform	AWS EC2
Operating System	Ubuntu Linux
Container Runtime	Docker
Kubernetes	Kind
Kubernetes CLI	kubectl
Package Manager	Helm
Application Language	Python
Application Framework	Flask
Container Registry	Docker Hub
Version Control	Git and GitHub

2.1 AWS EC2 Instance Setup

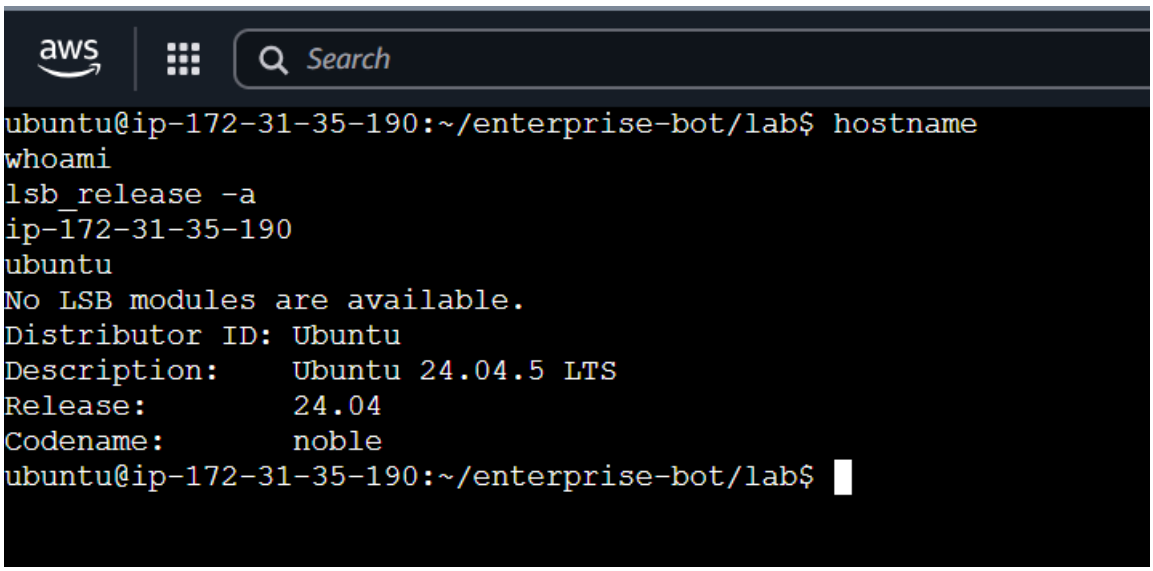
Created an Ubuntu EC2 instance to provide the Linux environment for completing the assignment.

Connected to the instance from Windows using SSH and a PEM private key.

Verified the environment using:

```
hostname
whoami
lsb_release -a
```

Screenshot 01 | EC2 Instance and Ubuntu Environment



2.2 Project Transfer and Extraction

Transferred the assignment ZIP file from the local Windows machine to the EC2 instance using SCP.

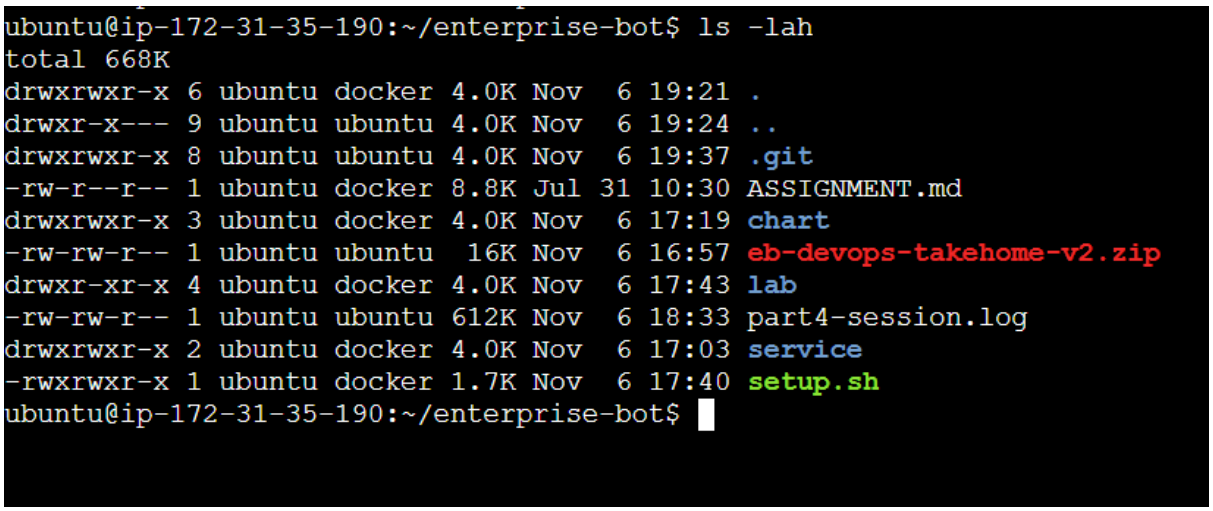
Extracted the assignment and navigated to the project directory:

```
cd ~/enterprise-bot
```

Inspected the project files:

```
ls -lah
```

Screenshot 02 | Project Directory Structure



2.3 Development Tools Installation

Installed and configured the required development tools:

- Docker
- kubectl
- Kind
- Helm

Verified the installations:

```
docker --version
kubectl version --client
kind version
helm version
```

Verified Docker functionality:

```
docker run hello-world
```

The Docker test container executed successfully.

Screenshot 03 | Installed Development Tools



3. Kubernetes Cluster Setup

Created a local Kubernetes cluster using Kind:

```
kind create cluster --name demo
```

Verified the cluster:

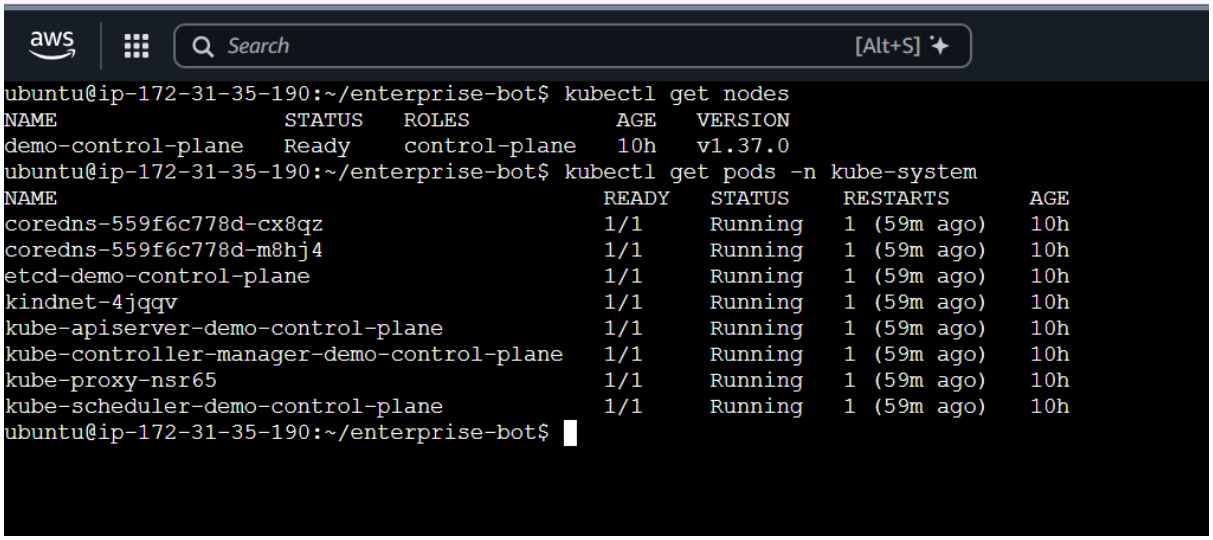
```
kubectl get nodes
```

Checked Kubernetes system components:

```
kubectl get pods -n kube-system
```

The Kind control-plane node and Kubernetes system components were running.

Screenshot 04 | Kind Cluster and Kubernetes Nodes



4. Part 1 – Python Application and Docker

4.1 Application Structure

Created the Python application under the service directory.

```
enterprise-bot/
└── service/
    ├── app.py
    ├── requirements.txt
    ├── Dockerfile
    └── .dockerignore
```

4.2 Python Flask Application

Developed a Flask application with two HTTP endpoints.

GET /

Returns application information in JSON format, including:

- Application name
- Application version
- Container hostname

The application name and version can be configured through environment variables.

GET /healthz

Provides a health-check endpoint that returns HTTP 200 when the application is healthy.

4.3 Application Dependencies

Created requirements.txt with the Flask dependency:

```
Flask==3.1.2
```

4.4 Dockerfile

Created a multi-stage Dockerfile using Python 3.12 slim Bookworm.

The Docker image includes:

- Separate build and runtime stages.
- Python virtual environment.
- Non-root application user.
- Pinned base image tag.
- Application port 8080.

4.5 Docker Ignore

Created .dockerignore to exclude unnecessary files from the Docker build context, including:

- Python cache files.
- Virtual environments.
- Git metadata.
- Environment files.
- Log files.

4.6 Docker Image Build

Built the application image:

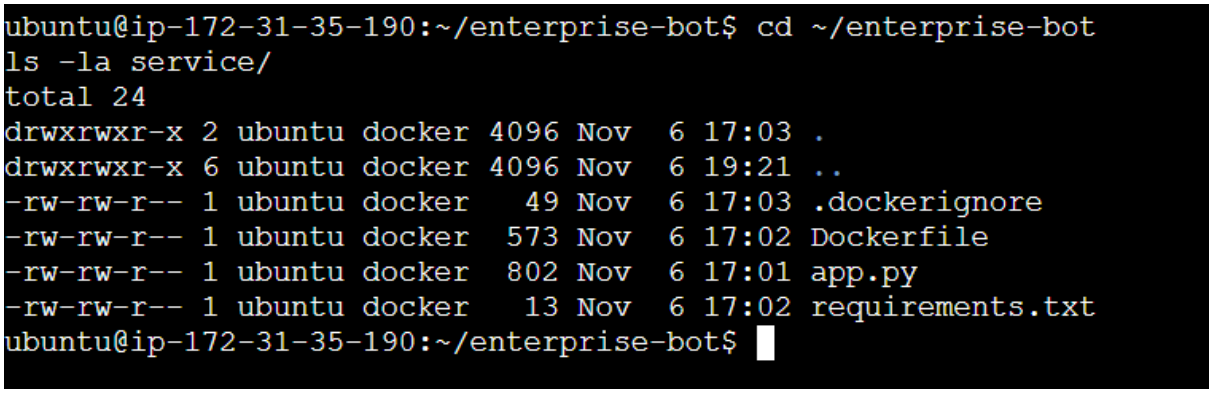
```
cd ~/enterprise-bot/service
docker build -t enterprise-bot:1.0 .
```

Verified the image:

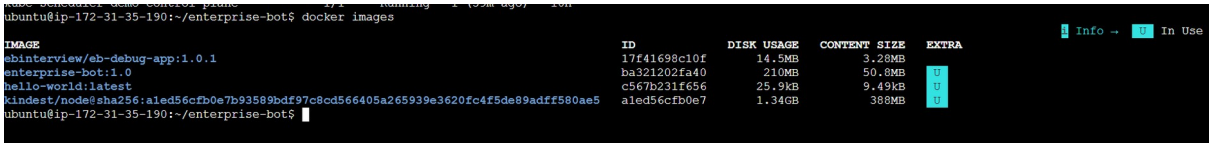
```
docker images
```

The Docker image was built successfully.

Screenshot 05 | Application Source Files



Screenshot 06 | Successful Docker Image Build



4.7 Container Deployment

Started the application container:

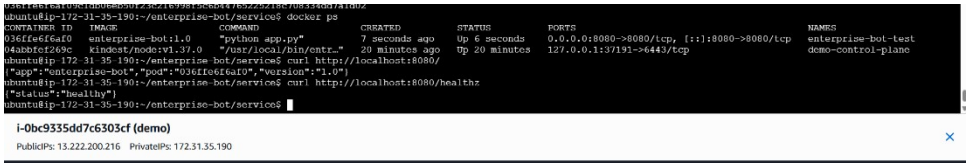
```
docker run -d \
  --name enterprise-bot-test \
  -p 8080:8080 \
  -e APP_NAME=enterprise-bot \
  -e VERSION=1.0 \
  enterprise-bot:1.0
```

Verified the running container:

```
docker ps
```

The application container was running with port 8080 mapped to the EC2 host.

Screenshot 07 | Running Docker Container



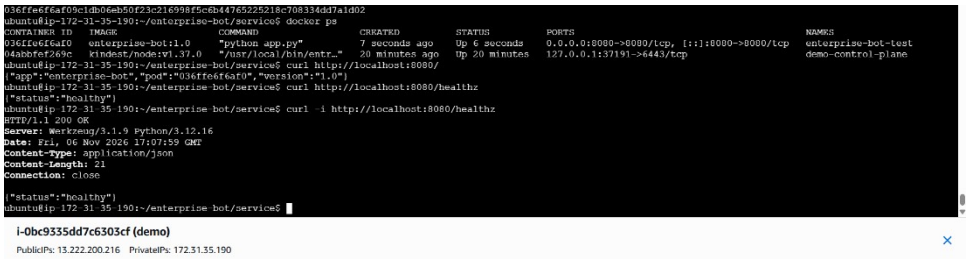
4.8 Application Endpoint Verification

The application endpoints were prepared for verification using:

```
curl -i http://localhost:8080/
curl -i http://localhost:8080/healthz
```

> **Note:** *Record the actual HTTP responses and capture the successful results here.*

Screenshot 08 | Application Endpoint Verification



5. Part 2 – Helm and Kubernetes Deployment

5.1 Helm Chart

Worked with the provided Helm chart under the assignment's lab/broken-chart/ directory.

The chart uses configurable values to manage application deployment settings, including:

- Container image repository and tag.
- Replica counts.
- Application environment variables.
- Container ports.
- Resource requests and limits.
- Kubernetes Deployment and Service resources.

5.2 Chart Validation

Validated the Helm chart using:

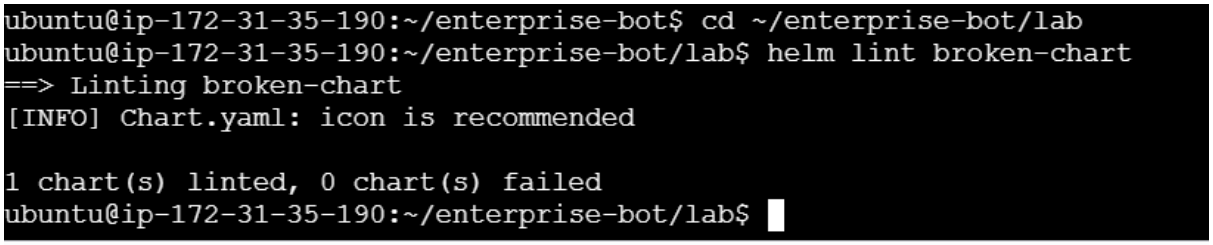
```
cd ~/enterprise-bot/lab
helm lint broken-chart
```

Rendered the chart templates for inspection:

```
helm template debug-lab broken-chart
```

The chart was linted and its rendered Kubernetes manifests were inspected during troubleshooting.

Screenshot 09 | Helm Chart Validation



5.3 Kubernetes Resource Inspection

Inspected application resources in the debug-lab namespace:

```
kubectl get pods -n debug-lab
kubectl get deployments -n debug-lab
kubectl get services -n debug-lab
```

Inspected pod details and events:

```
kubectl describe pod <pod-name> -n debug-lab
```

Screenshot 10 | Kubernetes Resources

```
ubuntu@ip-172-31-35-190:~/enterprise-bot/lab$ kubectl get pods -n debug-lab
kubectl get deployments -n debug-lab
kubectl get services -n debug-lab
NAME                         READY   STATUS      RESTARTS      AGE
backend-64b799c9c4-km7xb     1/1     Running     1 (68m ago)   9h
gateway-69ccbcdc95-qk6d8     1/1     Running     1 (68m ago)   9h
metrics-758cd6c755-d4jsm     1/1     Running     1 (68m ago)   9h
migrate-vzwnj                0/1     Completed   0             10h
reporter-875f68874-9qxhh     0/1     Running     0             42m
reporter-api-check4          0/1     Completed   0             9h
reporter-api-test            0/1     Completed   0             28m
reporter-go-test             0/1     Completed   0             20m
reporter-gzip-test           0/1     Completed   0             27m
reporter-http-test           0/1     Error       0             22m
reporter-json-test           0/1     Completed   0             21m
reporter-ns-test             0/1     Error       0             21m
reporter-timeout-test        0/1     Completed   0             23m
worker-77478d7895-t94qb      1/1     Running     1 (68m ago)   9h
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
backend    1/1     1            1           10h
gateway    1/1     1            1           10h
metrics    1/1     1            1           10h
reporter   0/1     1            0           10h
worker     1/1     1            1           10h
NAME       TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
backend    ClusterIP   10.96.199.230   <none>        8080/TCP   10h
gateway    ClusterIP   10.96.66.95     <none>        80/TCP     10h
reporter   ClusterIP   10.96.76.1      <none>        8080/TCP   10h
ubuntu@ip-172-31-35-190:~/enterprise-bot/lab$
```

i-0bc9335dd7c6303cf (demo)

6. Part 3 – Deployment Automation

Worked with the assignment's deployment automation and Helm-based deployment workflow.

The deployment process involved:

1. Checking the Kubernetes environment.
2. Validating the Helm chart.
3. Applying the required deployment configuration.
4. Monitoring Kubernetes workloads.
5. Checking deployment and service status.
6. Verifying application health.

6.1 Deployment Script (setup.sh)

Created setup.sh in the project root to automate the end-to-end environment setup and application deployment. The script performs the following stages:

- Creates a Kind cluster named demo with ports 80 and 443 mapped to the host, or reuses the cluster if it already exists.
- Exports the kubeconfig and waits for all nodes to become Ready.
- Installs the ingress-nginx controller and waits for its rollout to complete.

- Builds the enterprise-bot:1.0 Docker image and loads it into the Kind cluster.
- Creates the demo namespace idempotently using a client-side dry run.
- Installs or upgrades the demo Helm release from the chart directory and waits for the resources to become ready.
- Displays the pods, services and ingress resources in the demo namespace.

The script uses set -euo pipefail so that it stops immediately on any failed command, unset variable or pipeline error.

setup.sh Bash

```
#!/bin/bash
set -euo pipefail

PROJECT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
CLUSTER_NAME="demo"
IMAGE_NAME="enterprise-bot:1.0"
NAMESPACE="demo"

echo "===== Enterprise Bot Setup ====="

# 1. Create or reuse kind cluster
if kind get clusters | grep -qx "$CLUSTER_NAME"; then
    echo "Kind cluster already exists. Reusing it."
else
    cat > /tmp/kind-config.yaml <<'CONFIG'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
    labels:
      ingress-ready: "true"
    extraPortMappings:
      - containerPort: 80
        hostPort: 80
        protocol: TCP
      - containerPort: 443
        hostPort: 443
        protocol: TCP
CONFIG

    kind create cluster --name "$CLUSTER_NAME" --config /tmp/kind-config.yaml
    rm -f /tmp/kind-config.yaml
fi

kind export kubeconfig --name "$CLUSTER_NAME"

kubectl wait --for=condition=Ready node --all --timeout=180s

# 2. Install ingress-nginx
kubectl apply -f
https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/
provider/kind/deploy.yaml

kubectl rollout status deployment/ingress-nginx-controller \
    -n ingress-nginx --timeout=180s

# 3. Build and load Docker image
docker build -t "$IMAGE_NAME" "$PROJECT_DIR/service"

kind load docker-image "$IMAGE_NAME" --name "$CLUSTER_NAME"

# 4. Create namespace
kubectl create namespace "$NAMESPACE" \
    --dry-run=client -o yaml | kubectl apply -f -
```

```
# 5. Install or upgrade Helm release
helm upgrade --install demo "$PROJECT_DIR/chart" \
    --namespace "$NAMESPACE" \
    --wait \
    --timeout 180s

echo "===== Setup Completed Successfully ====="

kubectl get pods -n "$NAMESPACE"
kubectl get services,ingress -n "$NAMESPACE"
```

Run the script from the project root:

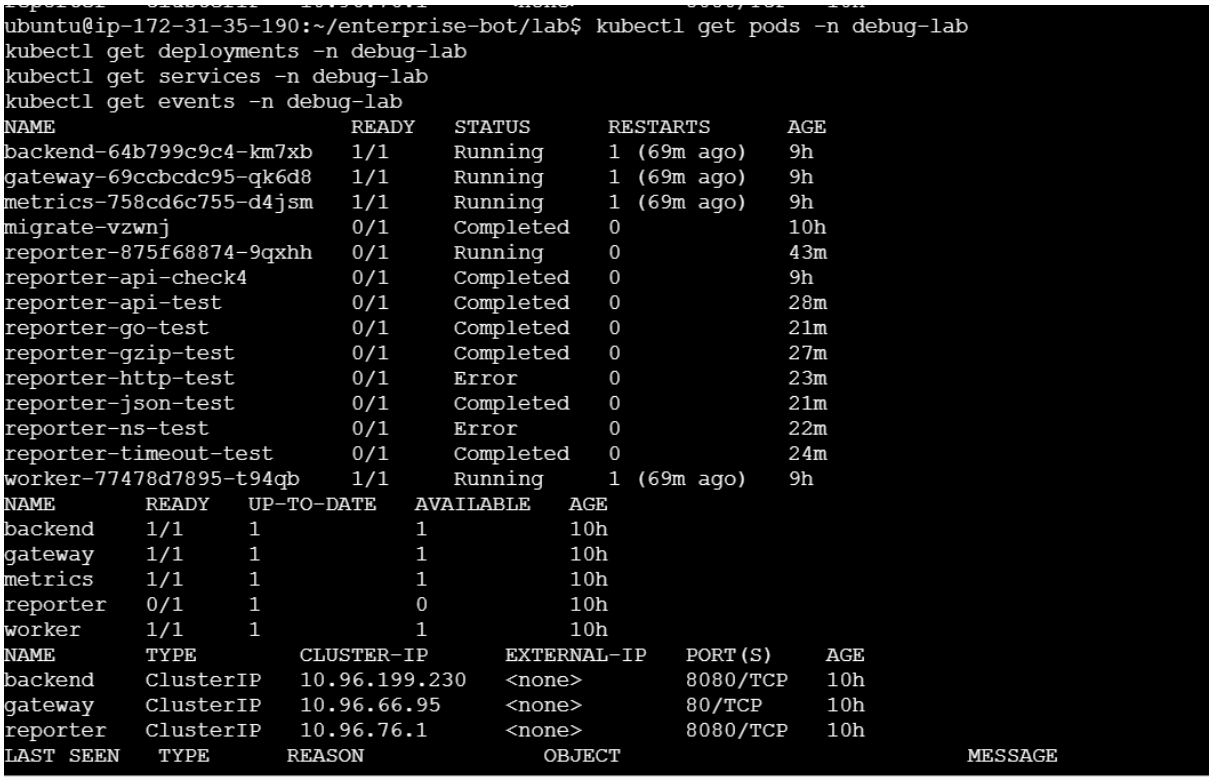
```
chmod +x setup.sh
./setup.sh
```

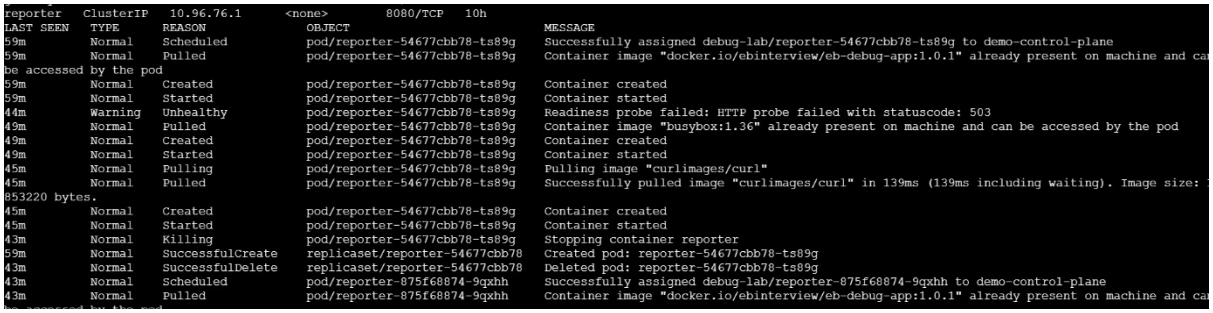
6.2 Deployment Verification

Useful Kubernetes verification commands:

```
kubectl get pods -n debug-lab
kubectl get deployments -n debug-lab
kubectl get services -n debug-lab
kubectl get events -n debug-lab
```

Screenshot 11 | Deployment Automation and Verification





```
reporter   ClusterIP   10.96.76.1      <none>        8080/TCP   10h
LAST SEEN   TYPE      REASON              OBJECT                                  MESSAGE
59m         Normal    Scheduled           pod/reporter-54677cbb78-ts89g           Successfully assigned debug-lab/reporter-54677cbb78-ts89g to demo-control-plane
59m         Normal    Pulled              pod/reporter-54677cbb78-ts89g           Container image "docker.io/ebinterview/eb-debug-app:1.0.1" already present on machine and ca
be accessed by the pod
59m         Normal    Created             pod/reporter-54677cbb78-ts89g           Container created
59m         Normal    Started             pod/reporter-54677cbb78-ts89g           Container started
44m         Warning   Unhealthy           pod/reporter-54677cbb78-ts89g           Readiness probe failed: HTTP probe failed with statuscode: 503
49m         Normal    Pulled              pod/reporter-54677cbb78-ts89g           Container image "busybox:1.36" already present on machine and can be accessed by the pod
49m         Normal    Created             pod/reporter-54677cbb78-ts89g           Container created
49m         Normal    Started             pod/reporter-54677cbb78-ts89g           Container started
45m         Normal    Pulling             pod/reporter-54677cbb78-ts89g           Pulling image "curlimages/curl"
45m         Normal    Pulled              pod/reporter-54677cbb78-ts89g           Successfully pulled image "curlimages/curl" in 139ms (139ms including waiting). Image size:
853220 bytes.
45m         Normal    Created             pod/reporter-54677cbb78-ts89g           Container created
45m         Normal    Started             pod/reporter-54677cbb78-ts89g           Container started
43m         Normal    Killing             pod/reporter-54677cbb78-ts89g           Stopping container reporter
59m         Normal    SuccessfulCreate    replicaset/reporter-54677cbb78          Created pod: reporter-54677cbb78-ts89g
43m         Normal    SuccessfulDelete    replicaset/reporter-54677cbb78          Deleted pod: reporter-54677cbb78-ts89g
43m         Normal    Scheduled           pod/reporter-875f68874-9qxhh            Successfully assigned debug-lab/reporter-875f68874-9qxhh to demo-control-plane
43m         Normal    Pulled              pod/reporter-875f68874-9qxhh            Container image "docker.io/ebinterview/eb-debug-app:1.0.1" already present on machine and ca
be accessed by the pod
```

7. Part 4 – Kubernetes Troubleshooting

7.1 Overview

The troubleshooting exercise involved investigating intentionally broken Kubernetes resources and correcting deployment configuration issues.

The investigation used:

- kubectl
- Helm
- Kubernetes Events
- Pod logs
- Deployment descriptions
- Service and endpoint inspection
- Kubernetes RBAC verification

The application image supplied by the assignment was retained:

```
docker.io/ebinterview/eb-debug-app:1.0.1
```

The supplied application image and tag were not intentionally replaced.

7.2 Issues Investigated and Configuration Changes

The following areas were investigated and addressed during troubleshooting:

1. Gateway configuration

Corrected the backend URL configuration to use the Kubernetes backend Service:

```
http://backend:8080
```

2. Migration Job configuration

Corrected the migration Job's restart policy to use:

```
restartPolicy: Never
```

3. Metrics workload resources

Reviewed and adjusted CPU and memory requests and limits to avoid unnecessarily high resource allocation.

4. Worker writable storage

Configured writable temporary storage using an emptyDir volume and appropriate filesystem group settings.

5. Reporter RBAC configuration

Inspected the reporter ServiceAccount, Role, and RoleBinding. Corrected the RoleBinding subject configuration and investigated the reporter's permission to list pods.

6. Reporter readiness and API response issue

Investigated the reporter's repeated readiness failures and JSON parsing errors.

The reporter logs showed:

```
pod list failed: parse pod list: unexpected end of JSON input
```

The reporter's /healthz endpoint returned HTTP 503, preventing the pod from becoming Ready.

7.3 Reporter Troubleshooting Evidence

Inspected reporter logs:

```
kubectl logs deployment/reporter -n debug-lab --tail=300
```

Inspected reporter pod configuration:

```
kubectl describe pod reporter-875f68874-9qxhh -n debug-lab
```

Checked reporter Service endpoints:

```
kubectl get endpoints reporter -n debug-lab -o wide
```

The reporter Service had no endpoints because the reporter pod was not Ready.

7.4 Kubernetes API Connectivity Tests

Created diagnostic pods using the reporter ServiceAccount to test Kubernetes API access.

Tested the cluster-wide pod API:

```
https://kubernetes.default.svc/api/v1/pods
```

The API returned HTTP 200 and a complete JSON response.

Also tested the namespace-specific API:

```
https://kubernetes.default.svc/api/v1/namespaces/debug-lab/pods
```

The API returned HTTP 200.

A separate Python diagnostic test successfully parsed the response:

```
JSON valid; pod count: 13
```

These tests confirmed that the reporter ServiceAccount could authenticate, access the Kubernetes API, and retrieve parseable pod-list JSON.

However, the reporter application continued to report the JSON parsing error. The exact application-level cause remained unresolved.

7.5 Verification Status

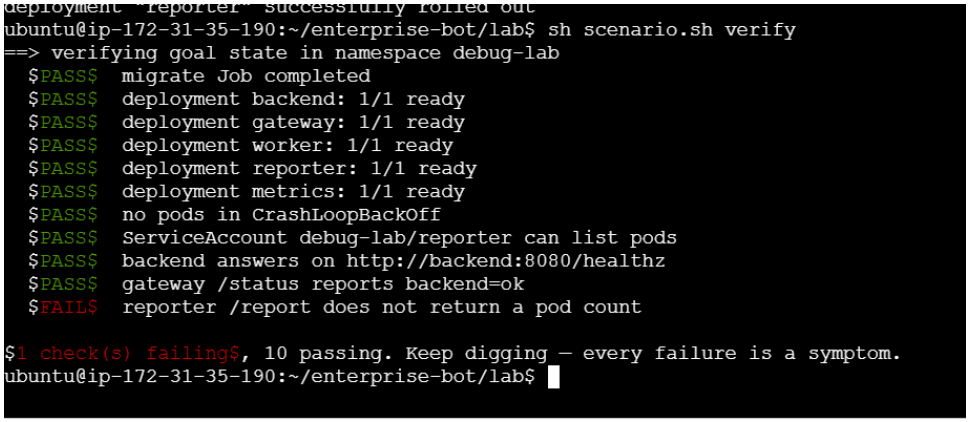
The last recorded scenario verification result was:

10/11 checks passed

One check remained failing: the reporter pod-count verification.

The other recorded checks passed.

Screenshot 12 | Scenario Verification Output



7.6 Troubleshooting Conclusion

Most of the Kubernetes configuration issues were addressed and the scenario verifier passed 10 out of 11 checks.

The remaining reporter issue requires further investigation of the application's internal HTTP response handling and JSON parsing behavior.

No claim is made that the reporter issue has been fully resolved.

8. Part 5 – Ingress to Gateway API Migration Strategy

The proposed migration strategy for approximately 40 ingress-nginx Ingress resources is to migrate gradually while keeping the existing ingress controller available during the transition.

The migration would follow these steps:

1. Audit the existing Ingress resources, annotations, TLS configurations, paths, and backend services.
2. Install a Kubernetes Gateway API-compatible controller.
3. Create the required GatewayClass and Gateway resources.
4. Convert existing Ingress routing rules into HTTPRoute resources.
5. Configure TLS, backend references, and hostname matching.
6. Test the new Gateway using a separate hostname or controlled test traffic.
7. Gradually shift production traffic to the new Gateway.
8. Monitor application availability, HTTP errors, latency, and routing behavior.
9. Keep the existing ingress-nginx controller and Ingress resources available for rollback.
10. Remove the old ingress controller only after confirming that all routes work correctly through Gateway API.

This staged migration reduces downtime risk and provides a rollback path if unexpected routing or connectivity problems occur.

9. Part 6 – Production Readiness Considerations

The following areas should be considered before deploying the application in a production environment.

9.1 Resource Management

Define appropriate CPU and memory requests and limits for each workload based on actual application requirements.

9.2 Health Checks

Configure readiness and liveness probes to ensure that traffic is routed only to healthy application instances.

9.3 Security

Use non-root containers, read-only root filesystems where possible, least-privilege RBAC, and secure handling of credentials.

9.4 Monitoring

Use monitoring and logging tools to track:

- Pod and node health.
- CPU and memory usage.
- Application availability.
- Deployment failures.
- Application errors.

9.5 High Availability

Use suitable replica counts, pod disruption budgets, resource planning, and autoscaling for production workloads.

9.6 Deployment Strategy

Use controlled rolling deployments and maintain a rollback strategy for failed releases.

9.7 Further Improvements

Before production use, the remaining reporter application issue must be resolved and all scenario verification checks must pass.

10. Git and GitHub

Used Git for source control and GitHub to maintain the assignment repository.

The project changes were committed and pushed to the GitHub repository.

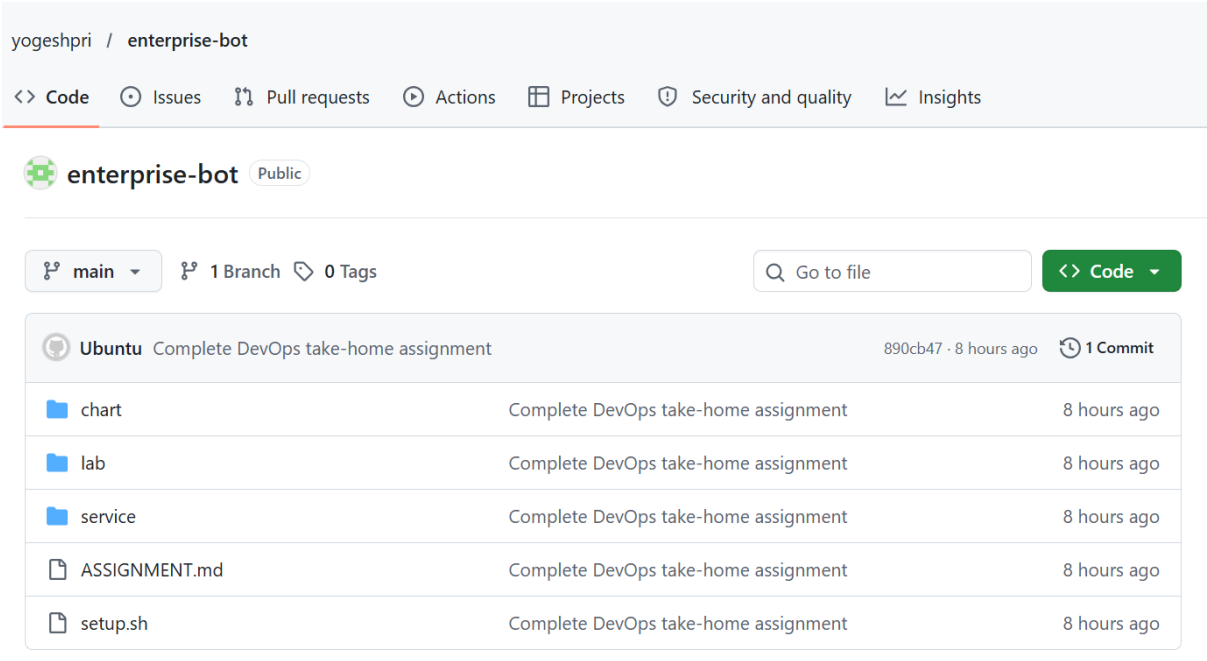
Repository:

https://github.com/yogeshpri/enterprise-bot.git

Useful commands:

```
git status
git add .
git commit -m "Update DevOps assignment"
git push origin main
```

Screenshot 13 | GitHub Repository



11. Final Project Status

Assignment Component	Status
AWS EC2 environment	Completed
Development tools setup	Completed
Kind cluster setup	Completed
Python Flask application	Created
Docker image build	Completed
Docker container deployment	Completed
Application endpoint verification	Record final test result
Helm chart validation	Completed
Kubernetes troubleshooting	Investigated and configuration fixes applied
Deployment automation	Worked on
Gateway API migration answer	Documented
Production readiness considerations	Documented
Scenario verification	10/11 checks passed in last recorded run
Reporter pod-count check	Pending investigation

12. Conclusion

This assignment provided practical experience in containerizing a Python application, managing Docker images, creating a Kubernetes environment using Kind, validating Helm charts, and troubleshooting Kubernetes deployments.

It also involved investigating application readiness failures, Kubernetes RBAC permissions, API connectivity, resource configuration, and service endpoints.

The work strengthened practical understanding of Docker, Kubernetes, Helm, deployment automation, and troubleshooting in a DevOps environment.

The remaining reporter verification issue has been documented transparently for further investigation.